HOWARD Help Configuration

Contents

1 Introduction

HOWARD Configuration JSON file defined default configuration regarding resources (e.g. threads, memory), settings (e.g. verbosity, temporary files), default folders (e.g. for databases) and paths to external tools.

Examples:

Example of a configuration JSON file

```
{
  "threads": 8,
  "memory": null,
  "verbosity": "WARNING",
  "folders": {
    "databases": {
      "genomes": "~/howard/databases/genomes/current",
      "annotations": [
        "~/howard/databases/annotations/current",
        "~/howard/databases/dbnsfp/current",
        "~/howard/databases/dbsnp/current"
      ],
      "parquet": [
        "~/howard/databases/annotations/current"
      ],
      "bcftools": [
        "~/howard/databases/annotations/current"
      ],
      "annovar": "~/howard/databases/annovar/current",
      "snpeff": "~/howard/databases/snpeff/current",
      "varank": "~/howard/databases/varank/current"
    }
  },
  "tools": {
    "bcftools": "bcftools",
    "bgzip": "bgzip",
    "java": "java",
    "snpeff": "~/howard/tools/snpeff/current/bin/snpEff.jar",
    "annovar": "~/howard/tools/annovar/current/bin/table_annovar.pl",
    "exomiser": "~/howard/tools/exomiser/current/bin/exomiser-cli-13.2.0.jar",
    "splice": {
       "docker": {
         "image": "bioinfochrustrasbourg/splice:0.2.6",
         "entrypoint": "/bin/bash",
         "options": null,
         "command": null
       }
    }
  }
}
```

2 folders

Folders configuration such as for databases.

2.1 databases

Default folders for databases that follows the specific database HOWARD format. These folders will be used in HOWARD tools to autodetect databases by their name and using assembly. Within database folders, multiple releases can be provides (e.g. 'b152' and 'b156' for dbSNP and 'hg19' assembly within folder '~/howard/databases/dbsnp/current/hg19', in 2 subfolders resp.)

Format: `/path/to/databases/<db_name>/<db_release>/<assembly>/<database_files>`

Examples:

Example of a configuration for databases folders

```
{
   "databases": {
      "genomes": "~/howard/databases/genomes/current",
      "annotations": [
         "~/howard/databases/annotations/current",
         "~/howard/databases/dbnsfp/current",
         "~/howard/databases/dbsnp/current"
      ],
      "parquet": [
         "~/howard/databases/annotations/current"
      ],
      "bcftools": [
         "~/howard/databases/bcftools/current"
      ],
      "annovar": "~/howard/databases/annovar/current",
      "snpeff": "~/howard/databases/snpeff/current",
      "exomiser": "~/howard/databases/exomiser/current"
   }
}
```

2.1.1 genomes

Genome folder with, for each assembly, FASTA files, indexes, and all files generated by pygenome module.

Type: `Path`

Format: `A folder path (without assembly)`

Default: `~/howard/databases/genomes/current`

Examples:

Path to genomes folder

```
{
   "genomes": "~/howard/databases/genomes/current"
}
```

2.1.2 annotations

Annotation databases folders that contains databases in various format such as Parquet, VCF, duckDB and TSV.

Type: `Path`

Format: `a list of folder path (without assembly)`

Default: `["~/howard/databases/annotations/current"]`

Examples:

Uniq folder with multiple annotations for Parquet annotation method, or other External tools

```
{
   "annotations": [
      "~/howard/databases/annotations/current"
   ]
}
```

Combinason of 2 folders with multiple annotations for Parquet annotation method, or other External tools

```
{
   "annotations": [
      "~/howard/databases/annotations/current",
      "~/howard/databases/dejavu/current",
      "~/howard/databases/dbnsfp/current"
   ]
}
```

2.1.3 parquet

Annotation databases folders that contains databases in Parquet format.

Format: `a list of folder path (without assembly)`

Default: `["~/howard/databases/annotations/current"]`

Examples:

Uniq folder with multiple annotations for Parquet annotation method

```
{
   "annotations": [
      "~/howard/databases/annotations/current"
   ]
}
```

Combinason of 2 folders with multiple annotations for Parquet annotation method

```
{
   "annotations": [
      "~/howard/databases/annotations/current",
      "~/howard/databases/dejavu/current",
      "~/howard/databases/dbnsfp/current"
   ]
}
```

2.1.4 bcftools

Annotation databases folders for BCFTools annotation.

Format: `a list of folder path (without assembly)`

Default: `["~/howard/databases/bcftools/current"]`

Examples:

Uniq folder with multiple VCF and BED files for BCFTools annotation

```
{
   "bcftools": [
      "~/howard/databases/bcftools/current"
   ]
}
```

Combinason of 2 folders with multiple VCF and BED files for BCFTools annotation

```
{
   "bcftools": [
      "~/howard/databases/bcftools/current",
      "~/howard/databases/dejavu/current"
   ]
}
```

2.1.5 annovar

Annotation databases folder for Annovar annotation.

Format: `a list of folder path (without assembly)`

Default: `["~/howard/databases/annovar/current"]`

Examples:

Uniq folder with multiple Annovar TXT files for Annovar annotation

```
{
   "annovar": "~/howard/databases/annovar/current/"
}
```

2.1.6 snpeff

Annotation databases folders for snpEff annotation.

Format: `A folder path (without assembly)`

Default: `~/howard/databases/snpeff/current`

Examples:

Path to snpEff database folder

```
{
   "snpeff": "~/howard/databases/snpeff/current/"
}
```

2.1.7 exomiser

Annotation databases folders for Exomiser annotation.

Format: `A folder path (without assembly)`

Default: `~/howard/databases/exomiser/current`

Examples:

Path to Exomiser database folder

```
{
   "exomiser": "~/howard/databases/exomiser/current/"
}
```

2.1.8 refseq

Annotation databases folders for refSeq annotation.

Type: `str`

Format: `A folder path (without assembly)`

Default: `~/howard/databases/refseq/current`

Examples:

Path to refSeq files folder

```
{
   "refseq": "~/howard/databases/refseq/current/"
}
```

3 tools

External tools paths that can be defined as path to a binary or a dict including the binary type (such as "bin", "jar", "perl").

External tools can be configured with docker, using 'docker' as binary type and options to define docker 'image' (mandatory), 'entrypoint', 'command' and docker runtime 'options'.

For docker-based annotation tools, structural settings are split into two sections:

- 'annotation' for annotation command semantics (primary flags, VCF update behavior, defaults such as parameters/resources/output pattern)
- 'runtime' for technical container mounts (databases, mounts, paths).

Within annotation.vcf_update, 'update_existing_fields' controls whether existing INFO fields are overwritten during update_from_vcf; this default can be overridden per entry in parameters JSON.

Examples:

Example of a configuration for tools, with env $PATH, full path and path with type

```
{
   "tools": {
      "bcftools": "bcftools",
      "bgzip": "bgzip",
      "java": "/usr/bin/java",
      "snpeff": "~/howard/tools/snpeff/current/bin/snpEff.jar",
      "annovar": {"jar": "~/howard/tools/annovar/current/bin/table_annovar.pl"},
      "exomiser": {"jar": "~/howard/tools/exomiser/current/bin/exomiser-cli-13.2.0.jar"}
   }
}
```

Example of a configuration for bcftools with a docker image (example with howard docker image)

```
{
   "tools": {
      "bcftools": {
         "docker": {
           "image": "howard:0.14.0",
           "entrypoint": "bcftools",
           "options": null,
           "command": null
         }
      }
   }
}
```

Example of a configuration for splice with a docker image

```
{
   "tools": {
      "splice": {
         "docker": {
           "image": "bioinfochrustrasbourg/splice:0.2.6",
           "entrypoint": "/bin/bash",
           "options": null,
           "command": null
         }
      }
   }
}
```

Example of a docker annotation tool configuration with annotation/runtime split

```
{
   "tools": {
      "vep": {
         "docker": {
           "image": "ensemblorg/ensembl-vep",
           "entrypoint": "vep",
           "command": null,
           "annotation": {
             "primary": {
```

```
                "input": "--input_file",
                "output": "--output_file",
                "threads": "--fork"
              },
              "vcf_update": {
                "remove_info": true,
                "add_samples": true,
                "update_header": true,
                "update_existing_fields": false
              },
              "defaults": {
                "parameters": ["--vcf", "--cache", "--offline", {"key": "--compress_output", "value": "bg
                "resources": {"threads": 4, "memory": null},
                "output_pattern": "*.vcf.gz"
              }
            },
            "runtime": {
              "databases": [{"name": "vep", "container_path": "/opt/vep/.vep", "mode": "ro"}],
              "mounts": [],
              "paths": []
            }
          }
        }
    }
}
```


3.1 bcftools

BCFTools binary (see https://samtools.github.io/bcftools/).

Default: `bcftools`

Examples:

Path to binary in $PATH env variable


```
{
    "bcftools": "bcftools"
}
```


Path to binary as a dict with binary type 'bin'


```
{
    "bcftools": {"bin": "~/howard/tools/bcftools/current/bin/bcftools"}
}
```


3.2 bgzip

BGZip binary (see https://samtools.github.io/bcftools/).

Default: `bgzip`

Examples:

Path to binary in $PATH env variable


```
{
    "bgzip": "bgzip"
}
```


Path to binary as a dict with binary type 'bin'


```
{
    "bgzip": {"bin": "~/howard/tools/htslib/current/bin/bgzip"}
}
```

3.3 java

Java binary (see https://www.java.com).

Default: `java`

Examples:

Path to binary in $PATH env variable

```
{
   "java": "java"
}
```

Path to binary as a dict with binary type 'bin'

```
{
   "java": {"bin": "/usr/bin/java"}
}
```

3.4 snpeff

snpEff binary (see https://pcingola.github.io/SnpEff/).

Default: `~/howard/tools/snpeff/current/bin/snpEff.jar`

Examples:

Path to binary as a dict without binary type

```
{
   "snpeff": "~/howard/tools/snpeff/current/bin/snpEff.jar"
}
```

Path to binary as a dict with binary type 'jar'

```
{
   "snpeff": {"jar": "~/howard/tools/snpeff/current/bin/snpEff.jar"}
}
```

3.5 annovar

ANNOVAR perl script (see https://annovar.openbioinformatics.org/).

Default: `~/howard/tools/annovar/current/bin/table_annovar.pl`

Examples:

Path to binary as a dict without binary type

```
{
   "annovar": "~/howard/tools/annovar/current/bin/table_annovar.pl"
}
```

Path to binary as a dict with binary type 'perl'

```
{
   "annovar": {"jar": "~/howard/tools/annovar/current/bin/table_annovar.pl"}
}
```

3.6 exomiser

Exomiser binary (see https://www.sanger.ac.uk/tool/exomiser/).

Default: `~/howard/tools/exomiser/current/bin/exomiser-cli-13.2.0.jar`

Examples:

Path to binary as a dict without binary type

```
{
   "snpeff": "~/howard/tools/exomiser/current/bin/exomiser-cli-13.2.0.jar"
}
```

Path to binary as a dict with binary type 'jar'

```
{
   "snpeff": {"jar": "~/howard/tools/exomiser/current/bin/exomiser-cli-13.2.0.jar"}
}
```

3.7 splice

Splice Docker image binary (see https://hub.docker.com/r/bioinfochrustrasbourg/splice).

Default: `None`

Examples:

Configuration of Docker image

```
{
   "splice": {
      "docker": {
        "image": "bioinfochrustrasbourg/splice:0.2.6",
        "entrypoint": "/bin/bash",
        "options": null,
        "command": null
      }
   }
}
```

3.8 docker_tool_template

Generic template to configure any docker-based tool in HOWARD.

Use this template under tools..docker and adapt values for your tool:

- docker.image: Docker image reference (required), for example '/:'.
- docker.entrypoint: command executed as container entrypoint. Usually the tool binary (for example 'vep') or a shell (for example '/bin/bash').
- docker.command: static command suffix defined in configuration. If null, command arguments are fully built at runtime from parameters/options.
- docker.options: extra docker run options (list of tokens), for example '--user', '--cpus', '--memory', '--env', '--network'.
- docker.config: optional docker runtime behavior for HOWARD wrapper.
- docker.config.automount: if true, HOWARD automatically mounts detected host paths needed by inputs/outputs/resources.
- docker.config.notremove: if true, keep container after execution (debug usage). If false, run with automatic removal.
- docker.config.tmp: container temporary folder used by HOWARD when staging temporary files.
- docker.annotation: annotation-specific semantics used by annotation_docker workflow.
- docker.annotation.primary: mapping of logical I/O/thread concepts to CLI flags for the tool.
- docker.annotation.primary.input: CLI flag used to pass input VCF path inside container.
- docker.annotation.primary.output: CLI flag used to pass output VCF path inside container.
- docker.annotation.primary.threads: CLI flag used to pass thread count to the tool.
- docker.annotation.primary.assembly: optional assembly mapping section to inject a mapped assembly value.
- docker.annotation.primary.assembly.flag: CLI flag used to pass assembly value (for example '--assembly').

- docker.annotation.primary.assembly.source: mapping source name. Use 'howard' for identity default mapping (e.g. use howard assembly such as 'hg19'), or a configured source such as 'GRCH' (e.g. assembly 'hg19' will be mapped as 'GRCH37' as defined on HOWARD).
- docker.annotation.primary.assembly.mapping: optional local source mapping override (for example {'hg19':'GRCh37','hg38':'GRC which is GRCh mapping).
- docker.annotation.primary.genome: optional genome section to inject fasta path for the selected assembly.
- docker.annotation.primary.genome.flag: CLI flag used to pass genome fasta path (for example '--fasta').
- docker.annotation.primary.genome.source: genome source. Default behavior uses 'howard' to resolve fasta from folders.databases.genomes.
- docker.annotation.primary.genome.path: optional explicit fasta path/folder (required when source is not 'howard').
- docker.annotation.primary.genome.mode: genome mount policy. With 'rw', HOWARD keeps fasta mount in ro and adds a rw mount on genome directory to allow lock files.
- docker.annotation.vcf_update: controls how HOWARD imports annotation output VCF back into the variants table.
- docker.annotation.vcf_update.remove_info: remove existing INFO fields before update/import step.
- docker.annotation.vcf_update.add_samples: include sample columns/genotypes when importing output VCF.
- docker.annotation.vcf_update.update_header: update VCF header metadata from output VCF before importing INFO values.
- docker.annotation.vcf_update.update_existing_fields: if true, overwrite already existing INFO fields; if false, keep existing values when possible. This default can be overridden per entry in parameters JSON.
- docker.annotation.defaults: default values applied to all entries using this tool, unless overridden in parameters JSON entry.
- docker.annotation.defaults.parameters: default tool arguments list (string flags or key/value objects).
- docker.annotation.defaults.resources: default resource limits for this tool.
- docker.annotation.defaults.resources.threads: default thread count passed to the tool (subject to HOWARD/global limits).
- docker.annotation.defaults.resources.memory: default memory limit for the tool (for example '8G' or null for no explicit value).
- docker.annotation.defaults.output_pattern: expected output file pattern generated by the tool inside working directory.
- docker.runtime: technical mounts/paths used to make host resources available in the container.
- docker.runtime.databases: database mount declarations with database 'name', destination 'container_path', and optional mode ('ro' or 'rw').
- docker.runtime.mounts: explicit mount objects for arbitrary host/container mappings (source/target/mode).
- docker.runtime.paths: legacy/compact mount declarations as strings ('/host:/container[:mode]'). Use mounts when possible for clarity.

Examples:

Generic docker tool template with all possible options

```
{
   "<tool_name>": {
      "docker": {
         "image": "<registry>/<image>:<tag>",
         "entrypoint": "<binary_or_shell>",
         "command": null,
         "options": ["--user", "1000:1000"],
         "config": {
            "automount": true,
            "notremove": false,
```

```
            "tmp": "/tmp"
         },
         "annotation": {
            "primary": {
               "input": "--input_file",
               "output": "--output_file",
               "threads": "--fork",
               "assembly": {"flag": "--assembly", "source": "GRCH"},
               "genome": {"flag": "--fasta", "source": "howard", "mode": "rw"}
            },
            "vcf_update": {
               "remove_info": true,
               "add_samples": true,
               "update_header": true,
               "update_existing_fields": false
            },
            "defaults": {
               "parameters": ["--flag", {"key": "--opt", "value": "value"}],
               "resources": {"threads": 4, "memory": null},
               "output_pattern": "*.vcf.gz"
            }
         },
         "runtime": {
            "databases": [{"name": "<database_name>", "container_path": "/data/db", "mode": "ro"}],
            "mounts": [{"source": "/host/path", "target": "/container/path", "mode": "rw"}],
            "paths": ["/host/path:/container/path:rw"]
         }
      }
   }
}
```

Example using this template with VEP

```
{
   "vep": {
      "docker": {
         "image": "ensemblorg/ensembl-vep",
         "entrypoint": "vep",
         "command": null,
         "annotation": {
            "primary": {"input": "--input_file", "output": "--output_file", "threads": "--fork", "assemb
            "vcf_update": {"remove_info": true, "add_samples": true, "update_header": true, "update_exis
            "defaults": {
               "parameters": ["--vcf", "--cache", "--offline", {"key": "--compress_output", "value": "bg
               "resources": {"threads": 4, "memory": null},
               "output_pattern": "*.vcf.gz"
            }
         },
         "runtime": {
            "databases": [{"name": "vep", "container_path": "/opt/vep/.vep", "mode": "ro"}],
            "mounts": [],
            "paths": []
         }
      }
   }
}
```

4 chunking

Chunking is a technique used to process large files by dividing them into smaller, more manageable pieces (chunks), by specifying chunk size and partitioning scheme. This approach allows each chunk to be processed independently, reducing memory usage, disk usage (duckDB swap), and can help prevent crashes or slowdowns due to memory constraints. Once all chunks are processed, the results are merged to produce the final output.

This method is particularly useful for handling large datasets or files that would otherwise be too resource-intensive to process in one go (such as low memory, or data particularly huge due to extensive annotation).

However, splitting, processing, and merging chunks can introduce additional computational overhead, may mislead with some calculations (due to not covering the full region of interest, such as all variants of a gene for compound heterozygote calculation), and can not apply full data features (such as statistics). Moreover, the order of data can change during the chunking-merging process.

To enable chunking, include the `chunking` section in the parameters JSON file. This section includes parameters to enable chunking, define chunk size, partitioning scheme, and sorting of the final output.

Examples:

Default configuration for chunking

```
{
   "chunking": {
      "chunking_enable": false,
      "chunking_size": 1000000
      "chunking_partitions": "None",
      "chunking_sort": false
}
```

Example of a configuration for chunking big files by chromosome with 100 million variants per chunk and sorting of final output

```
{
   "chunking": {
      "chunking_enable": true,
      "chunking_size": 100000000
      "chunking_partitions": "#CHROM",
      "chunking_sort": false
}
```

4.1 chunking_enable

Chunking process aims to improve processing efficiency, either by:

- splitting input file into smaller parts.
- using duckDB database as a intermediate file storage.

Default: `False`

Examples:

Enable chunking

```
{
   "chunking_enable": true
}
```

Disable chunking

```
{
   "chunking_enable": false
}
```

4.2 chunking_size

Chunking size is the number of variants to process at a time. With 'parquet' chunking mode, input file will be split into multiple smaller Parquet files with a maximum of 'chunking_size' variants per file. With 'duckdb' chunking mode, the global chunk_size parameter will replaceded by 'chunking_size'.

Type: `int`

Default: `1000000`

Examples:

Set chunking size to 1 million variants

```
{
   "chunking_size": 1000000
}
```

Set chunking size to 10 million variants

```
{
   "chunking_size": 10000000
}
```

4.3 chunking_partitions

Partitioning scheme to use with 'parquet' chunking mode, defining how the input file is partitioned during chunking.

Type: `str`

Default: `None`

Examples:

No partitioning

```
{
   "chunking_partitions": "None"
}
```

Partition by chromosome

```
{
   "chunking_partitions": "#CHROM"
}
```

4.4 chunking_sort

Sort final output after chunking is applied. Prevents issues with unordered variants after chunked processing.

Default: `False`

Examples:

Enable sorting

```
{
   "chunking_sort": true
}
```

Disable sorting

```
{
   "chunking_sort": false
}
```

5 threads

Specify the number of threads to use for processing HOWARD. It determines the level of parallelism, either on python scripts, duckdb engine and external tools. It and can help speed up the process/tool. Use -1 to use all available CPU/cores. Either non valid value is 1 CPU/core.

Type: `int`

Default: `-1`

Examples:

Automatically detect all available CPU/cores

```
{
   "threads": -1
}
```

Define 8 CPU/cores

```
{
   "threads": 8
}
```

6 memory

Specify the memory to use in format FLOAT[kMG] (e.g. '8G', '12.42G', '1024M'). It determines the amount of memory for duckDB engine and external tools (especially for JAR programs). It can help to prevent 'out of memory' failures. By default (None) is 80%% of RAM (for duckDB).

Type: `str`

Format: `FLOAT[kMG]`

Default: `None`

Examples:

Automatically detect all available CPU/cores

```
{
   "threads": -1
}
```

Define 8 CPU/cores

```
{
   "threads": 8
}
```

7 assembly

Genome Assembly (e.g. 'hg19', 'hg38').

Type: `str`

Default: `hg19`

Examples:

Default assembly for all analysis tools

```
{
   "assembly": "hg19"
}
```

List of assemblies for databases download tool

```
{
   "assembly": "hg19,hg38"
}
```

8 verbosity

Verbosity level Available: CRITICAL, ERROR, WARNING, INFO, DEBUG or NOTSET

- DEBUG: Detailed information, typically of interest only when diagnosing problems.
- INFO: Confirmation that things are working as expected.
- WARNING: An indication that something unexpected happened.
- ERROR: Due to a more serious problem.
- CRITICAL: A serious error.
- FATAL: A fatal error.
- NOTSET: All messages.

Type: `str`

Choices: `['CRITICAL', 'ERROR', 'WARNING', 'INFO', 'DEBUG', 'NOTSET', 'WARN', 'FATAL']`

Default: `INFO`

Examples:

Default verbosity

```
{
   "verbosity": "INFO"
}
```

ERROR level (quiet mode)

```
{
   "verbosity": "ERROR"
}
```

For debug

```
{
   "verbosity": "DEBUG"
}
```

9 tmp

Temporary folder (e.g. '/tmp'). By default, '.tmp' for duckDB (see doc),external tools and python scripts.

Type: `Path`

Default: `None`

Examples:

System temporary folder

```
{
   "tmp": "/tmp"
}
```

HOWARD work directory

```
{
   "tmp": "~/howard/tmp"
}
```

Current work directory

```
{
   "tmp": ".tmp"
}
```

10 access

Access mode to variants input file in database. Either 'RW' for Read and Write, or 'RO' for Read Only. Read Only Access mode is usefull to not load data into database (especially input Parquet file) when it's not needed (e.g. queries, fast annotation mode), reducing memory impact

Type: `str`

Choices: `['RW', 'RO', None, 'None']`

Default: `None`

Examples:

Default mode (auto detection)

```
{
   "access": None
}
```

Read and Write mode

```
{
   "access": "RW"
}
```

Read only mode

```
{
   "access": "RO"
}
```

11 duckdb_settings

DuckDB settings (see duckDB doc) as JSON (string or file). These settings have priority (see options 'threads', 'tmp'...). Examples: '{"TimeZone": "GMT", "temp_directory": "/tmp/duckdb", "threads": 8}'.

Type: `Path`

Default: `None`

Examples:

DuckDB settings JSON file

```
{
   "duckdb_settings": "/path/to/duckdb_config.json"
}
```

JSON string for Time zone, temporary directory and threads for duckDB

```
{
   "duckdb_settings": {
      "TimeZone": "GMT",
      "temp_directory": "/tmp/duckdb",
      "threads": 8
   }
}
```

12 chunk_size

Number of records in batch to export output file. The lower the chunk size, the less memory consumption. For Parquet partitioning, files size will depend on the chunk size.

Type: `int`

Default: `1048576`

Examples:

Chunk size of 1048576 by default

```
{
   "chunk_size": 1048576
}
```

Smaller chunk size to reduce Parquet file size and memory usage

```
{
   "chunk_size": 100000
}
```

13 log

Logs file (e.g. 'my.log').

Type: `Path`

Default: `None`

Examples:

Relative path to log file

```
{
   "log": "my.log"
}
```

HOWARD work directory

```
{
   "log": "~/howard/log"
}
```

Full path to log file

```
{
   "log": "/tmp/my.log"
}
```